Lecture9: Classification1

STA 4724

HanQin Cai

University of Central Florida

Classification

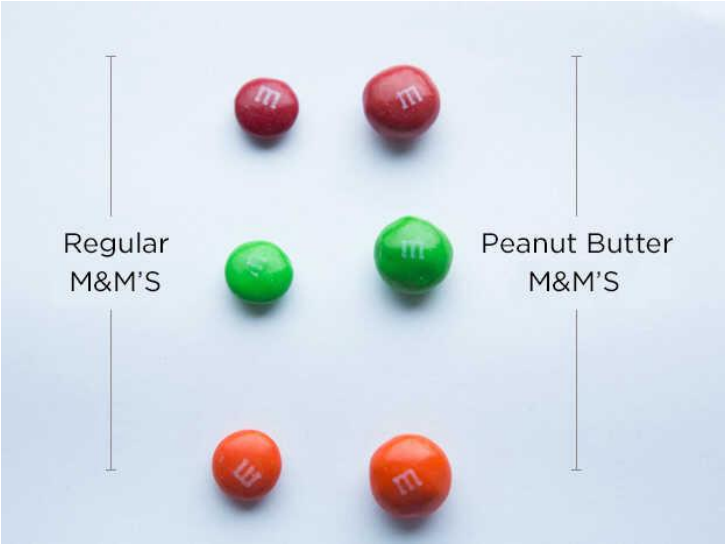
- A task that arranges objects systematically into appropriate groups (or categories) depending on the characteristics that define such groupings

- The groups are pre-defined according to established criteria
  - Training dataset is pre-labelled
  - Supervised learning

- No labelled data? Clustering first.

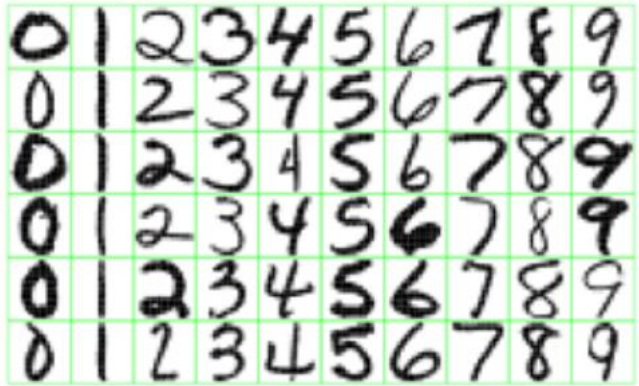
Classification is Natural to Human

- Human are good on finding low-dimension characteristics
- With the labels, we easily figure out the bigger M&Ms are peanut butter ones and the smaller ones are regular
- We also find the color doesn't matter much to the taste of M&Ms
- Classification is pattern recognition; more precisely, finding the major difference among different groups



Applications

- Two examples:
  - Spam email
    - ❖ Known as spam filter. Has been around for decades
    - ❖ A classic example we will refer to later
  - Handwritten Digit Recognition
    - ❖ The goal is to identify images of single digits 0 - 9 correctly
    - ❖ Helps USPS a lot
    - ❖ Multi-class classification



Confusion Matrices

- Before we talk about how to train a classifier, let's talk about how to evaluate the performance of a classifier.

- Use World War II air reconnaissance troop as an example
  - Detecting correctly when an enemy aircraft is flying above them
  - A binary classification system

		Predicted Class	
		Enemy Aircraft	Flock of Birds
Actual class	Enemy Aircraft	20	4
	Flock of Birds	6	70

Confusion Matrices

		Predicted Class	
		Enemy Aircraft	Flock of Birds
Actual class	Enemy Aircraft	20	4
	Flock of Birds	6	70

		Predicted Class	
		Class 1	Class 2
Actual class	Class 1	True Positives (TP)	False Negatives (FN)
	Class 2	False Positives (FP)	True Negatives (TN)

Confusion Matrices

		Predicted Class	
		Class 1	Class 2
Actual class	Class 1	True Positives (TP)	False Negatives (FN)
	Class 2	False Positives (FP)	True Negatives (TN)

- *Recall* or *True Positive Rate*
  - Also known as *sensitivity* or *hit rate*
  - An evaluation that counts the rate of positive data points that are correctly classified

$$TPR = \frac{TP}{TP + FN}$$

  - The higher the True Positive Rate, the fewer positives will be missed. (FP is not evaluated)
  - In our example, $TPR = 20/24 = 0.833$

Confusion Matrices

- *True Negative Rate* or *Specificity*
  - An evaluation that counts the rate of negatives data points that are correctly classified

$$TNR = \frac{TN}{TN + FP}$$

  - In our example, $TNR = 70/76 = 0.921$

- *Fallout* or *False Positive Rate*
  - Opposite to *TNR*
  - An evaluation that counts the rate of negative data that are mistakenly considered as positive

$$FPR = \frac{FP}{FP + TN} = 1 - TNR$$

  - In our example, $TNR = 6/76 = 1 - 0.921 = 0.079$

Confusion Matrices

- *Precision* or *Positive Predictive Value*
  - An evaluation that counts the rate of positive results that are true positive results

$$PPV = \frac{TP}{TP + FP}$$

  - In our example, $PPV = 20/26 = 0.769$

- *Accuracy*
  - An evaluation that counts the rate of points that have been correctly classified among all the data points (positive or negative)

$$ACC = \frac{TP + TN}{TP + FP + FN + TN}$$

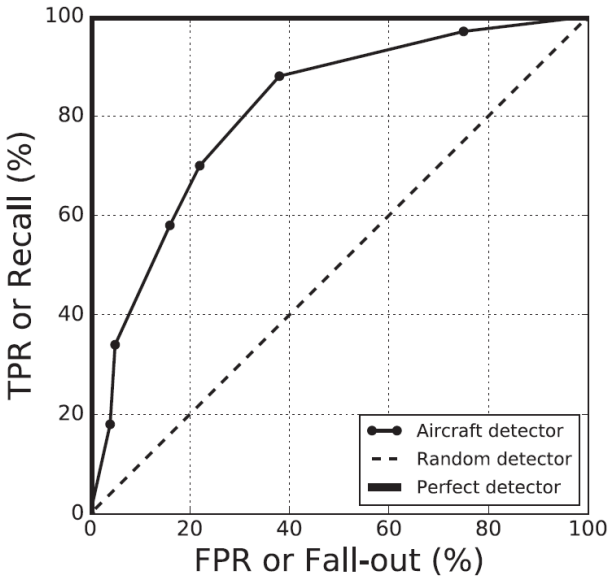
  - In our example, $ACC = 90/100 = 0.9$

Receiver Operating Characteristic (ROC) Curve

- The name origins in the British efforts during World Word II to differentiate enemy aircraft
  - Each radar receiver operator use different radar setting
  - Each of them was said to have their own characteristic and thus the name

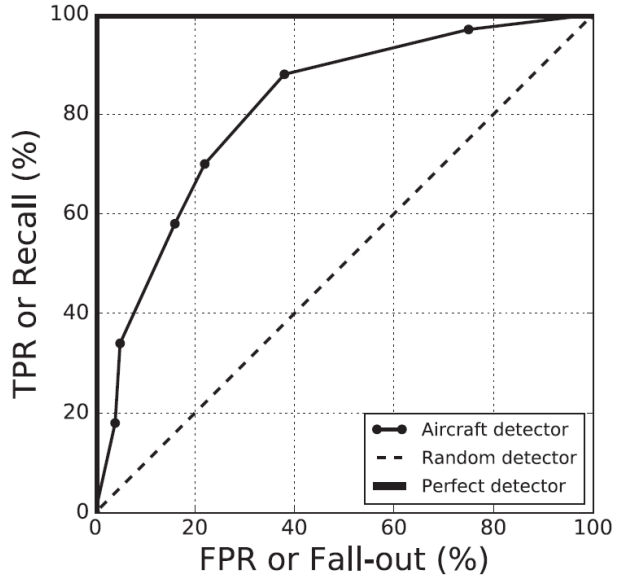
- ROC analyze a classifier is by plotting TPR as a function of FPR for different *cut-off points* or *thresholding* for the classifier. (Different settings on radar)

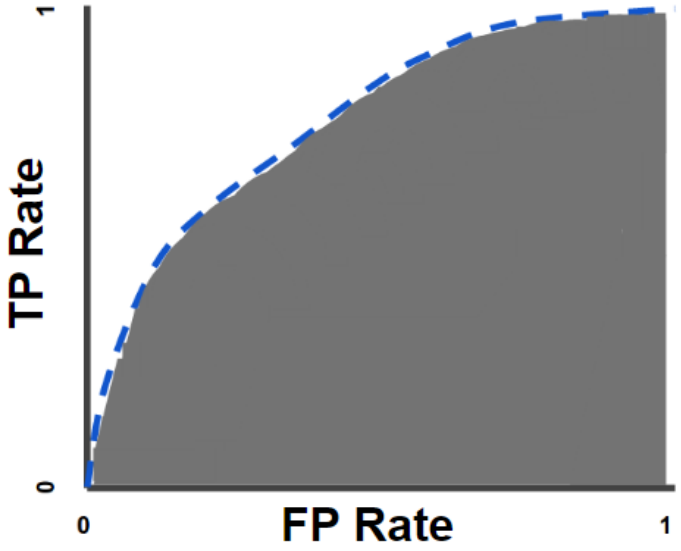
Setting	Enemy Aircraft Detected (%)	Flocks Detected (%)	Flocks Incorrectly Detected (%)
	Sensitivity	Specificity	Fallout
Off	0	100	0
S_1	18	96	4
S_2	34	95	5
S_3	58	84	16
S_4	70	78	22
S_5	88	62	38
S_6	97	25	75
Full blast	100	0	100



Area Under the ROC Curve (AUC)

- Measure how good a classifier is under different settings
  - A perfect classifier has AUC=1, a random classifier has AUC=0.5
  - A successful classifier should has AUC>0.5, and larger is better
  - ROC only applies to binary classifier





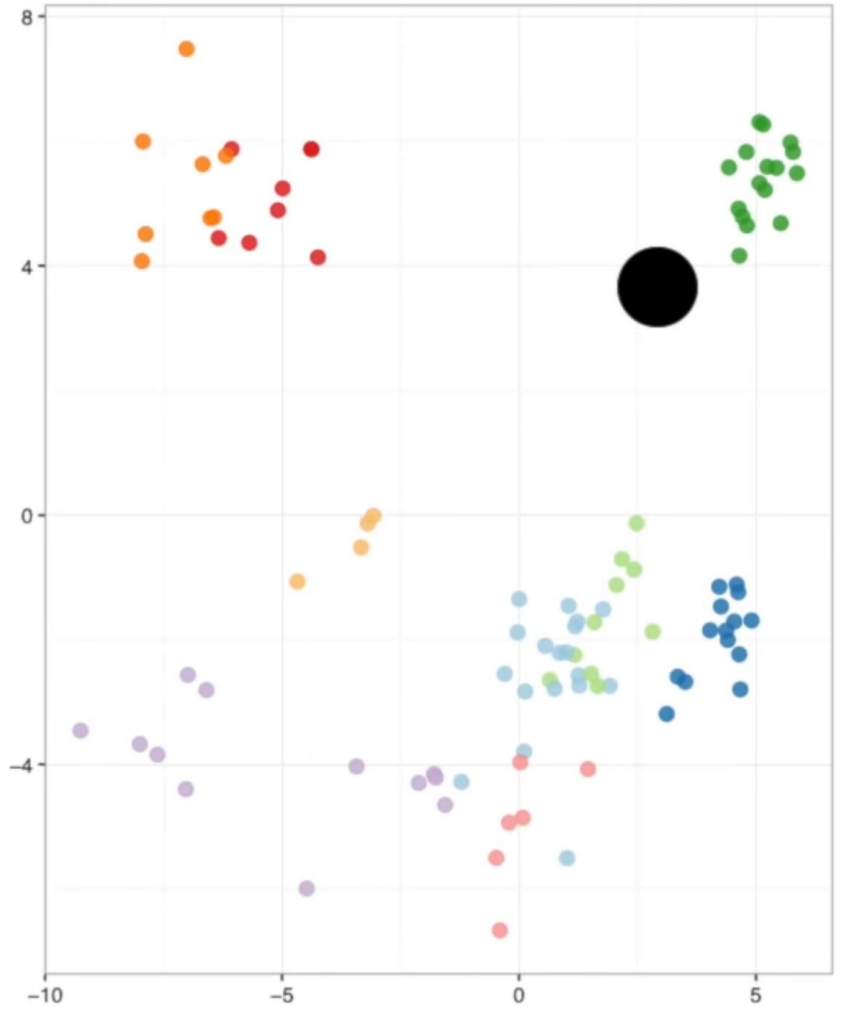
K-Nearest-Neighbors (KNN) classification

1. Choose a value for *K* as an input
2. Select the *K* nearest data points to the new observation
3. Find the most common class among the *K* points chosen
4. Assign this class to the new observation

Toy Example

- If $K = 1$, then we only use the nearest neighbor to define the category for the new point, which is green

- If $K = 11$, we would use the 11 neatest neighbors. In this example, all 11 nearest neighbors are green, thus the point is also assigned to green.

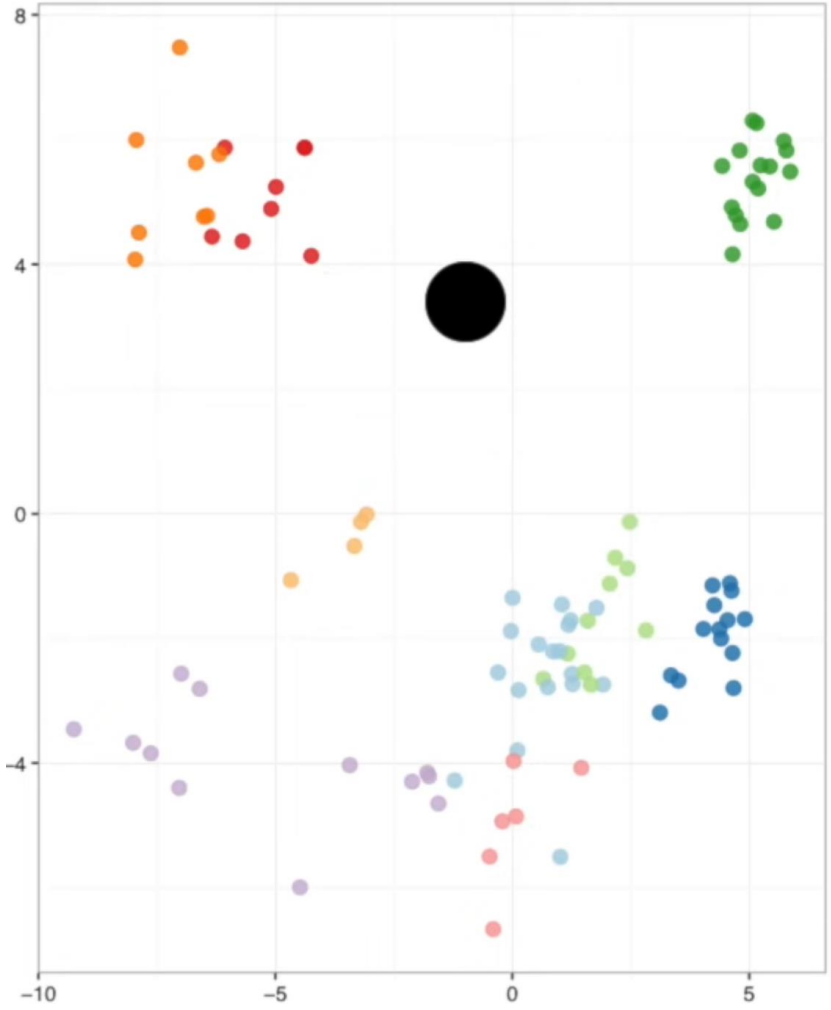


Toy Example

- Take $K = 11$ again. In this case,
  - 7 nearest neighbors are red
  - 3 nearest neighbors are orange
  - 1 nearest neighbors are green

- Since red got the most votes, the new point is assigned to red.

- In the case we have a tie vote, say 5 red vs 5 orange, we can flip a coin or decide not to assign the new point to any category



Picking the *K*

- Using odd *K* can avoid a lot of ties
- Small *K* can be noisy and subject to the effects of outliers
- Large *K* smooth over things, but you don't want *K* to be too large since the categories with only a few samples will always be out voted
- We can pick an "optimal" *K* via cross validation.

Time to Play

- Let’s see how to use *KNN* in **python**!